

Звіт
Лабораторна робота №3
Студента ПМІ-44
Стегня Віктора
Варіант 15

Задана вибірка №3:

	q0	x11	x12	x21	x22	x31	x32	y1	y2	y3	y4	y5
1	1	0	1	0	1	0	0.5	2.184	0.9472	1.592	3.08471	1.5208
2	2	0.1	0.9	0.19	0.91	0.18	0.6	2.22499	0.97999	1.61249	3.10882	1.56998
3	3	0.2	0.8	0.29	0.81	0.28	0.7	2.23031	0.98424	1.61515	3.11194	1.57637
4	4	0.3	0.7	0.39	0.71	0.38	0.8	2.24923	0.99938	1.62461	3.12307	1.59907
5	5	0.4	0.6	0.49	0.61	0.48	0.9	2.28175	1.0254	1.64087	3.1422	1.6381
6	6	0.5	0.5	0.59	0.51	0.58	1	2.32787	1.06229	1.66393	3.16933	1.69344
7	7	0.6	0.4	0.69	0.41	0.68	0.4	2.17759	0.94207	1.58879	3.08093	1.5131
8	8	0.7	0.3	0.79	0.31	0.78	0.3	2.19091	0.95272	1.59545	3.08877	1.52909
9	9	0.8	0.2	0.89	0.21	0.88	0.2	2.21783	0.97426	1.60891	3.1046	1.56139
10	10	0.9	0.1	0.99	0.11	0.98	0.1	2.25835	1.00668	1.62917	3.12844	1.61002
11	11	1	0	1	0	1	0	2.194	0.9552	1.597	3.09059	1.5328
12	12	2	1.5	1.8	1.1	2	1.2	6.4772	4.38176	3.7386	5.61012	6.67264
13	13	1.2	1.3	1.4	1.5	1.6	1.7	5.5256	3.62048	3.2628	5.05035	5.53072
14	14	0.8	0.1	0.7	0.6	0.5	0.6	1.9552	0.76416	1.4776	2.95012	1.24624
15	15	0.7	0.2	0.6	0.7	0.4	0.7	1.9804	0.78432	1.4902	2.96494	1.27648
16	16	0.6	0.3	0.5	0.8	0.3	0.8	2.0192	0.81536	1.5096	2.98776	1.32304
17	17	0.5	0.4	0.4	0.9	0.2	0.9	2.0716	0.85728	1.5358	3.01859	1.38592
18	18	0.4	0.5	0.3	1	0.1	1	2.1376	0.91008	1.5688	3.05741	1.46512
19	19	0.3	0.6	0.2	1.8	0	1.1	2.7128	1.37024	1.8564	3.39576	2.15536
20	20	0.2	0.7	0.1	0	1.1	1.2	2.8096	1.44768	1.9048	3.45271	2.27152
21	21	0.1	0.8	0	0.1	0.3	0.5	1.8916	0.71328	1.4458	2.91271	1.16992
22	22	0	0.9	1.1	0.2	0.4	0.4	2.2064	0.96512	1.6032	3.09788	1.54768
23	23	0.9	1	1.5	0.3	0.5	0.3	2.7948	1.43584	1.8974	3.444	2.25376

24	24	1.8	1.1	1.9	0.4	0.6	0.2	3.3968	1.91744	2.1984	3.79812	2.97616
25	25	0.1	1.2	1.9	1.3	0.7	0.1	3.2652	1.81216	2.1326	3.72071	2.81824
26	26	0	1.3	0.2	0.5	0.8	0	2.6704	1.33632	1.8352	3.37082	2.10448
27	27	0.7	0.6	0.5	0.4	0.3	0.2	2.0156	0.81248	1.5078	2.98565	1.31872
28	28	0.6	0.5	0.4	0.3	0.4	0.4	1.9644	0.77152	1.4822	2.95553	1.25728
29	29	0.5	0.4	0.3	0.2	0.5	0.6	1.9268	0.74144	1.4634	2.93341	1.21216
30	30	0.4	0.3	0.2	0.1	0.6	0.8	1.9028	0.72224	1.4514	2.91929	1.18336
31	31	0.3	0.2	0.1	0	0.7	1	1.8924	0.71392	1.4462	2.91318	1.17088
32	32	0.2	0.1	0	1.5	0.8	1.2	2.5004	1.20032	1.7502	3.27082	1.90048
33	33	0.1	0	0.4	1.3	0.9	1.4	2.4804	1.18432	1.7402	3.25906	1.87648
34	34	0	0.3	1.3	1.1	1	1.6	2.9472	1.55776	1.9736	3.53365	2.43664
35	35	0.5	0.4	0.5	0.9	1.1	1.8	3.1608	1.72864	2.0804	3.65929	2.69296
36	36	0.4	0.5	0.4	0.7	1.2	2	3.3012	1.84096	2.1506	3.74188	2.86144
37	37	0.3	0.6	0.3	0.5	1.3	0.8	3.0484	1.63872	2.0242	3.59318	2.55808
38	38	0.2	0.7	0.25	0.3	1.4	0.3	3.0424	1.63392	2.0212	3.58965	2.55088
39	39	0.1	0.8	0.14	0.1	1.5	0.4	3.2312	1.78496	2.1156	3.70071	2.77744
40	40	0	0.9	0.03	0	1.6	0.1	3.3316	1.86528	2.1658	3.75976	2.89792

Умови задачі (згідно тому, що сказано у прикріпленому відео поясненні лабораторної):

- b_{iq_0} визначають як середнє арифметичне:

$$b_{iq_0} = \frac{\left\{ \max_{i \in [1, m]} \bar{Y}_i[q_0] + \min_{i \in [1, m]} \bar{Y}_i[q_0] \right\}}{2}, q_0 = \overline{1, k_0}$$

- метод розв'язання несумісної системи рівнянь – метод найменших квадратів.

Задання вхідних даних

Вибірка задана csv файлом, вказаним вище, інші параметри прикріплюю:

```
N1, N2, N3, M = 2, 2, 2, 5
P1, P2, P3 = 3, 3, 3
SIZE = 40

data = load_and_normalize("data.csv", n1=N1, n2=N2, n3=N3, m=M, sample_size=SIZE)
results = fit_model(data["x1"], data["x2"], data["x3"], data["y"], p1=P1, p2=P2, p3=P3)

print_results(data, results)
plot_results(data["y"], results["y_pred"], data["y_names"])
```

✓ 0.5s

Результати

1. Формуємо вектор b на нормалізованих даних

Вектор b			
		b	
q1		0.218593	q21 0.184536
q2		0.223367	q22 0.221202
q3		0.223985	q23 0.289736
q4		0.226190	q24 0.359854
q5		0.229978	q25 0.344526
q6		0.235349	q26 0.275246
q7		0.217846	q27 0.198979
q8		0.219397	q28 0.193015
q9		0.222532	q29 0.188635
q10		0.227253	q30 0.185840
q11		0.219758	q31 0.184629
q12		0.753829	q32 0.255445
q13		0.618100	q33 0.253116
q14		0.191943	q34 0.307487
q15		0.194878	q35 0.332365
q16		0.199397	q36 0.348719
q17		0.205501	q37 0.319274
q18		0.213188	q38 0.318575
q19		0.280184	q39 0.340566
q20		0.291460	q40 0.352259

2. Знаходимо коефіцієнти λ для кожної групи змінних

Lambda для X1				
	T0	T1	T2	T3
X11	0.067401	0.050537	0.003901	0.003964
X12	0.067401	0.084137	0.003409	0.000809

Lambda для X2				
	T0	T1	T2	T3
X21	0.067401	0.021471	-0.002765	-0.001462
X22	0.067401	0.049629	0.010682	-0.001048

Lambda для X3				
	T0	T1	T2	T3
X31	0.067401	0.145318	0.029939	0.001072
X32	0.067401	0.035028	-0.000052	0.000597

3. Знаходимо матриці A1, A2, A3

Матриця A1		
	X11	X12
Y1	4.247648	2.530998
Y2	1.081159	1.393256
Y3	2.476185	1.361545
Y4	6.200944	2.497982
Y5	2.744116	2.395815

Матриця A2		
	X21	X22
Y1	3.957813	1.890107
Y2	0.945799	1.079926
Y3	2.316589	1.010779
Y4	5.876242	1.802385
Y5	2.494327	1.829235

Матриця A3		
	X31	X32
Y1	1.794815	4.782591
Y2	1.528275	1.292278
Y3	0.883353	2.776634
Y4	0.908090	6.862105
Y5	2.247642	3.165834

4. Виразимо частинні функції $\Phi_{ik}(x)$ через поліноми Чебишева

Для Y1:

$$\begin{aligned}\Phi_{11}(x) &= 0.28629488 T^*(X_{11}^0) + 0.21466227 T^*(X_{11}^1) + 0.01656987 T^*(X_{11}^2) + 0.01683725 T^*(X_{11}^3) \\ &\quad 0.17059130 T^*(X_{12}^0) + 0.21295168 T^*(X_{12}^1) + 0.00862829 T^*(X_{12}^2) + 0.00204687 T^*(X_{12}^3) \\ \Phi_{12}(x) &= 0.26675979 T^*(X_{21}^0) + 0.08497721 T^*(X_{21}^1) + -0.01094182 T^*(X_{21}^2) + -0.00578548 T^*(X_{21}^3) \\ &\quad 0.12739471 T^*(X_{22}^0) + 0.09380479 T^*(X_{22}^1) + 0.02019007 T^*(X_{22}^2) + -0.00198138 T^*(X_{22}^3) \\ \Phi_{13}(x) &= 0.12097196 T^*(X_{31}^0) + 0.26081888 T^*(X_{31}^1) + 0.05373580 T^*(X_{31}^2) + 0.00192488 T^*(X_{31}^3) \\ &\quad 0.32235051 T^*(X_{32}^0) + 0.16752559 T^*(X_{32}^1) + -0.00024758 T^*(X_{32}^2) + 0.00285562 T^*(X_{32}^3)\end{aligned}$$

Для Y2:

$$\begin{aligned}\Phi_{21}(x) &= 0.07287096 T^*(X_{11}^0) + 0.05463823 T^*(X_{11}^1) + 0.00421755 T^*(X_{11}^2) + 0.00428561 T^*(X_{11}^3) \\ &\quad 0.09390660 T^*(X_{12}^0) + 0.11722501 T^*(X_{12}^1) + 0.00474968 T^*(X_{12}^2) + 0.00112675 T^*(X_{12}^3) \\ \Phi_{22}(x) &= 0.06374763 T^*(X_{21}^0) + 0.02030702 T^*(X_{21}^1) + -0.00261477 T^*(X_{21}^2) + -0.00138256 T^*(X_{21}^3) \\ &\quad 0.07278790 T^*(X_{22}^0) + 0.05359606 T^*(X_{22}^1) + 0.01153575 T^*(X_{22}^2) + -0.00113208 T^*(X_{22}^3) \\ \Phi_{23}(x) &= 0.10300700 T^*(X_{31}^0) + 0.22208592 T^*(X_{31}^1) + 0.04575575 T^*(X_{31}^2) + 0.00163903 T^*(X_{31}^3) \\ &\quad 0.08710057 T^*(X_{32}^0) + 0.04526617 T^*(X_{32}^1) + -0.00006690 T^*(X_{32}^2) + 0.00077160 T^*(X_{32}^3)\end{aligned}$$

Для Y3:

$$\begin{aligned}\Phi_{31}(x) &= 0.16689686 T^*(X_{11}^0) + 0.12513832 T^*(X_{11}^1) + 0.00965948 T^*(X_{11}^2) + 0.00981535 T^*(X_{11}^3) \\ &\quad 0.09176922 T^*(X_{12}^0) + 0.11455689 T^*(X_{12}^1) + 0.00464157 T^*(X_{12}^2) + 0.00110111 T^*(X_{12}^3) \\ \Phi_{32}(x) &= 0.15614000 T^*(X_{21}^0) + 0.04973891 T^*(X_{21}^1) + -0.00640447 T^*(X_{21}^2) + -0.00338636 T^*(X_{21}^3) \\ &\quad 0.06812731 T^*(X_{22}^0) + 0.05016431 T^*(X_{22}^1) + 0.01079711 T^*(X_{22}^2) + -0.00105959 T^*(X_{22}^3) \\ \Phi_{33}(x) &= 0.05953870 T^*(X_{31}^0) + 0.12836706 T^*(X_{31}^1) + 0.02644711 T^*(X_{31}^2) + 0.00094737 T^*(X_{31}^3) \\ &\quad 0.18714738 T^*(X_{32}^0) + 0.09726051 T^*(X_{32}^1) + -0.00014374 T^*(X_{32}^2) + 0.00165789 T^*(X_{32}^3)\end{aligned}$$

Для Y4:

$$\begin{aligned}\Phi_{41}(x) &= 0.41794861 T^*(X_{11}^0) + 0.31337549 T^*(X_{11}^1) + 0.02418959 T^*(X_{11}^2) + 0.02457992 T^*(X_{11}^3) \\ &\quad 0.16836601 T^*(X_{12}^0) + 0.21017381 T^*(X_{12}^1) + 0.00851574 T^*(X_{12}^2) + 0.00202017 T^*(X_{12}^3) \\ \Phi_{42}(x) &= 0.39606347 T^*(X_{21}^0) + 0.12616733 T^*(X_{21}^1) + -0.01624553 T^*(X_{21}^2) + -0.00858982 T^*(X_{21}^3) \\ &\quad 0.12148220 T^*(X_{22}^0) + 0.08945122 T^*(X_{22}^1) + 0.01925303 T^*(X_{22}^2) + -0.00188942 T^*(X_{22}^3) \\ \Phi_{43}(x) &= 0.06120602 T^*(X_{31}^0) + 0.13196187 T^*(X_{31}^1) + 0.02718774 T^*(X_{31}^2) + 0.00097390 T^*(X_{31}^3) \\ &\quad 0.46251140 T^*(X_{32}^0) + 0.24036721 T^*(X_{32}^1) + -0.00035523 T^*(X_{32}^2) + 0.00409727 T^*(X_{32}^3)\end{aligned}$$

Для Y5:

$$\begin{aligned}\Phi_{51}(x) &= 0.18495565 T^*(X_{11}^0) + 0.13867869 T^*(X_{11}^1) + 0.01070467 T^*(X_{11}^2) + 0.01087740 T^*(X_{11}^3) \\ &\quad 0.16147985 T^*(X_{12}^0) + 0.20157771 T^*(X_{12}^1) + 0.00816745 T^*(X_{12}^2) + 0.00193754 T^*(X_{12}^3) \\ \Phi_{52}(x) &= 0.16811964 T^*(X_{21}^0) + 0.05355507 T^*(X_{21}^1) + -0.00689585 T^*(X_{21}^2) + -0.00364618 T^*(X_{21}^3) \\ &\quad 0.12329190 T^*(X_{22}^0) + 0.09078376 T^*(X_{22}^1) + 0.01953984 T^*(X_{22}^2) + -0.00191757 T^*(X_{22}^3) \\ \Phi_{53}(x) &= 0.15149286 T^*(X_{31}^0) + 0.32662276 T^*(X_{31}^1) + 0.06729319 T^*(X_{31}^2) + 0.00241053 T^*(X_{31}^3) \\ &\quad 0.21337976 T^*(X_{32}^0) + 0.11089348 T^*(X_{32}^1) + -0.00016389 T^*(X_{32}^2) + 0.00189028 T^*(X_{32}^3)\end{aligned}$$

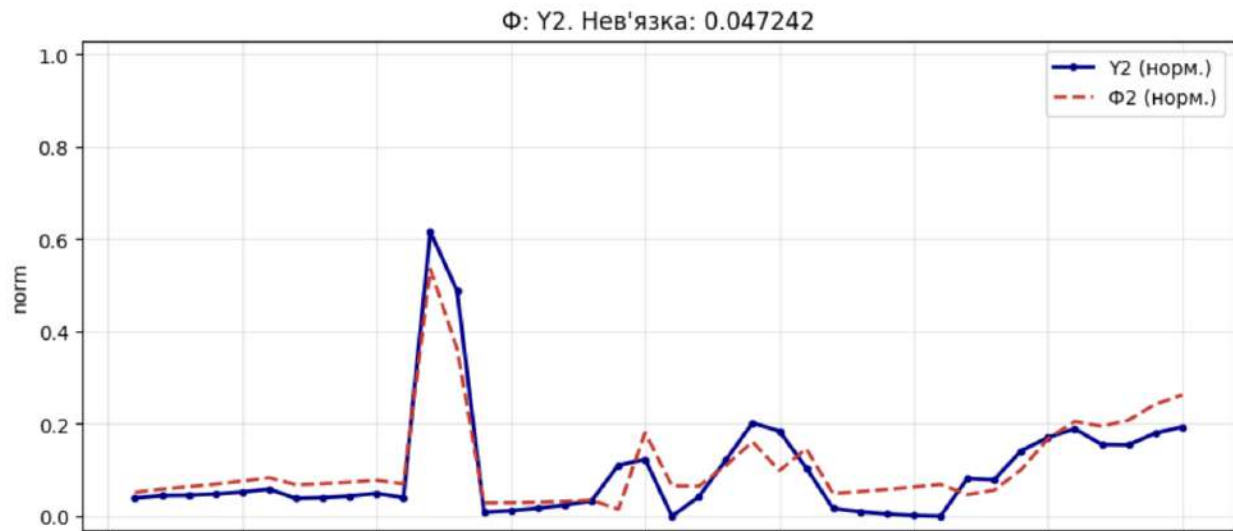
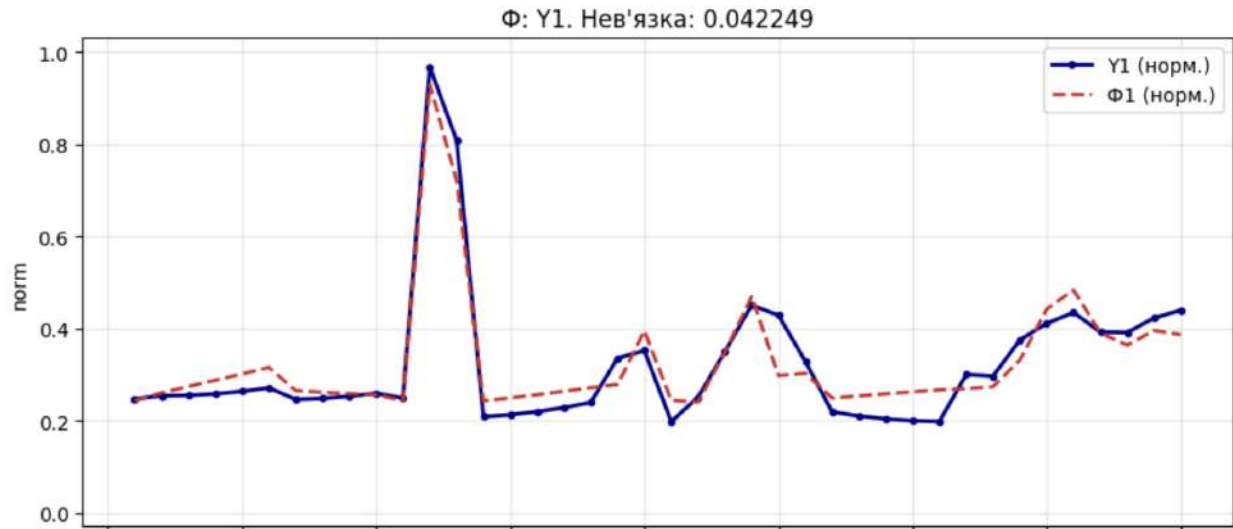
5. Тепер виразимо всі функції $\Phi_i(x)$ через $\Phi_{i1}, \Phi_{i2}, \Phi_{i3}$

$$\begin{aligned}\Phi_1(x) &= 0.522231052645461*\Phi_{11} + -0.010360590245463*\Phi_{12} + 0.567503350475319*\Phi_{13} \\ \Phi_2(x) &= 0.683550131715977*\Phi_{21} + -0.375091517633666*\Phi_{22} + 0.751177508083059*\Phi_{23} \\ \Phi_3(x) &= 0.481138580438729*\Phi_{31} + 0.064586576907557*\Phi_{32} + 0.532081045861249*\Phi_{33} \\ \Phi_4(x) &= 0.266682709424206*\Phi_{41} + 0.415449236355440*\Phi_{42} + 0.375748019196424*\Phi_{43} \\ \Phi_5(x) &= 0.646024443236168*\Phi_{51} + -0.259245609352413*\Phi_{52} + 0.686217857447620*\Phi_{53}\end{aligned}$$

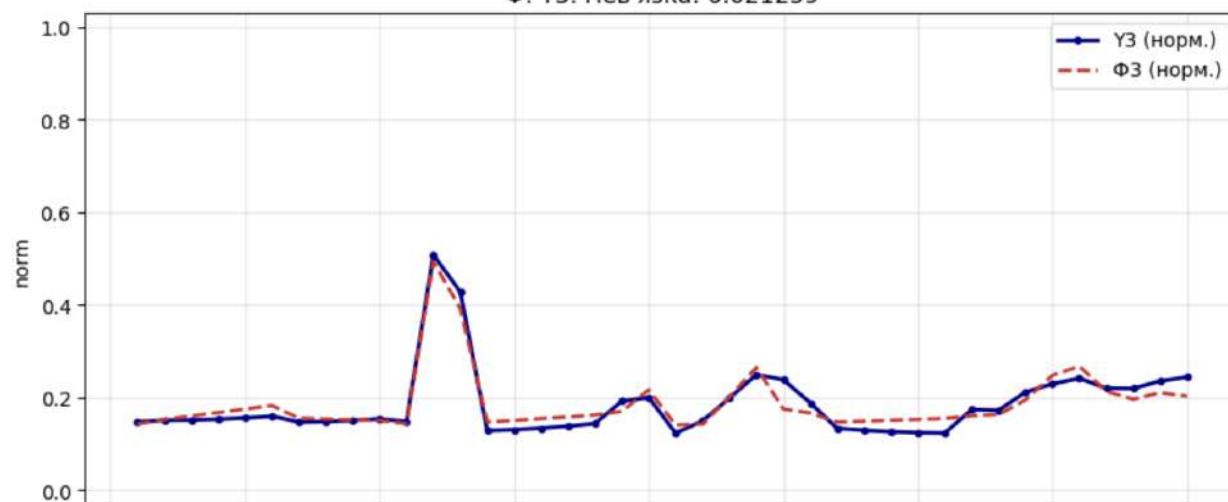
Матриця С

	c1	c2	c3
Y1	0.522231	-0.010361	0.567503
Y2	0.683550	-0.375092	0.751178
Y3	0.481139	0.064587	0.532081
Y4	0.266683	0.415449	0.375748
Y5	0.646024	-0.259246	0.686218

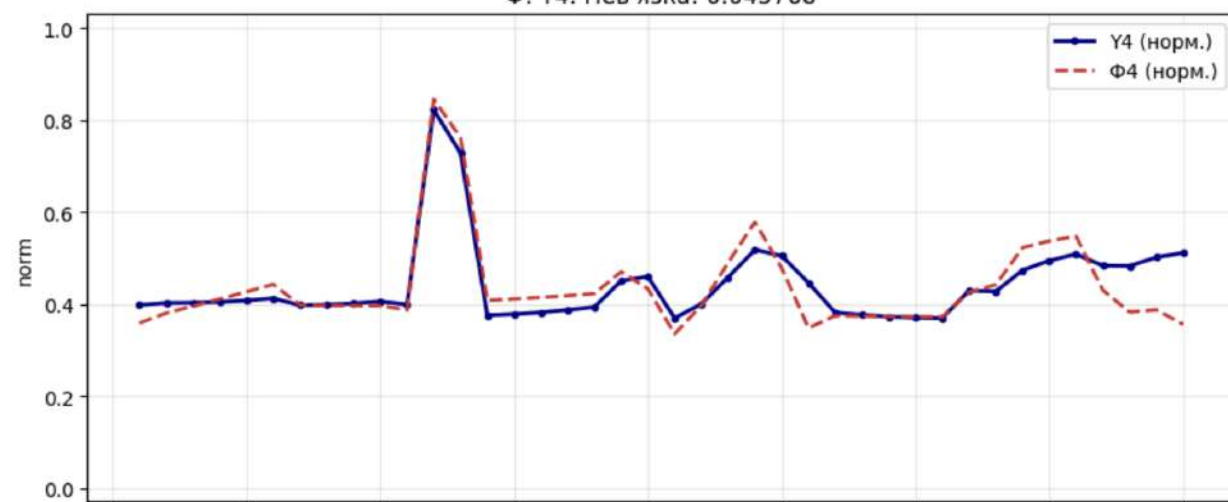
6. Порівняємо реальні та відновлені значення



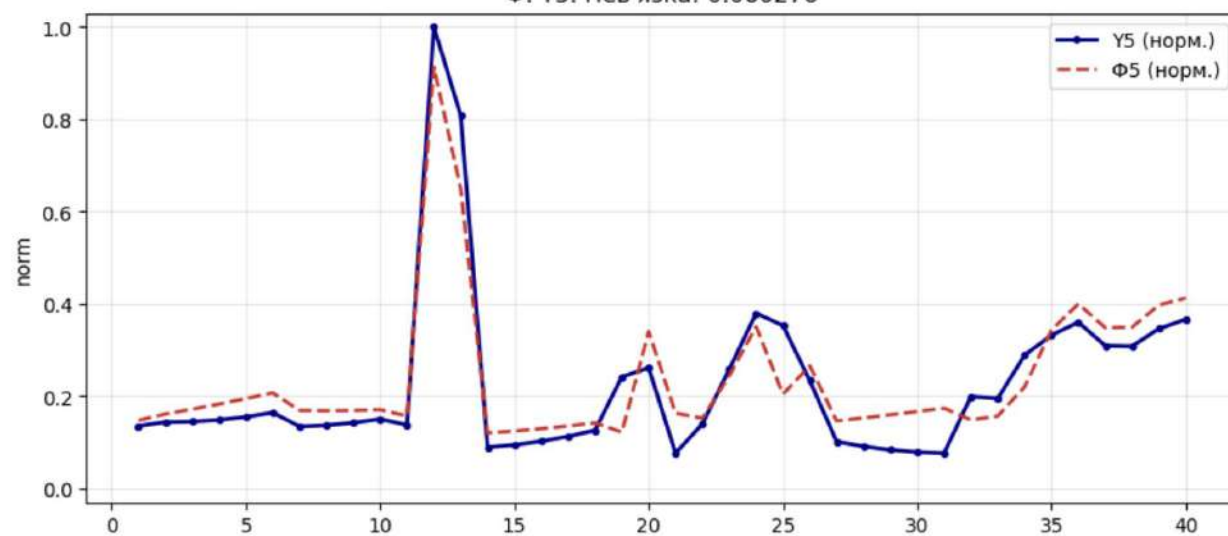
Ф: Y3. Нев'язка: 0.021259



Ф: Y4. Нев'язка: 0.045768



Ф: Y5. Нев'язка: 0.060276



Код реалізації

```
def read_numeric_csv(file_path, sample_size):
    for delimiter in (",", ";"):
        try:
            with open(file_path, newline="", encoding="utf-8") as file:
                reader = csv.reader(file, delimiter=delimiter)
                header = next(reader)
                rows = []
                for row in reader:
                    if not row:
                        continue
                    rows.append([float(value) for value in row[1:]])
                    if len(rows) == sample_size:
                        break
                if rows:
                    return header[1:], np.array(rows, dtype=float)
            except ValueError:
                continue
    raise ValueError("Не вдалося прочитати файл з числовими даними")

def load_and_normalize(file_path, n1, n2, n3, m, sample_size):
    column_names, matrix = read_numeric_csv(file_path, sample_size)
    n_total_x = n1 + n2 + n3

    x_matrix = matrix[:, :n_total_x]
    y_matrix = matrix[:, n_total_x:n_total_x + m]

    x_min = x_matrix.min(axis=0)
    x_max = x_matrix.max(axis=0)
    x_diff = np.where(x_max - x_min == 0, 1, x_max - x_min)
    x_norm = (x_matrix - x_min) / x_diff

    # Для Y використовуємо спільний діапазон, щоб зберегти відмінності між виходами.
    y_min = float(y_matrix.min())
    y_max = float(y_matrix.max())
    y_diff = y_max - y_min if y_max != y_min else 1.0
    y_norm = (y_matrix - y_min) / y_diff

    return {
        "column_names": column_names,
        "x1_names": column_names[:n1],
        "x2_names": column_names[n1:n1 + n2],
        "x3_names": column_names[n1 + n2:n_total_x],
        "y_names": column_names[n_total_x:n_total_x + m],
        "x1": x_norm[:, :n1],
        "x2": x_norm[:, n1:n1 + n2],
    }
```



```

    "x3": x_norm[:, n1 + n2:n_total_x],
    "y": y_norm,
    "y_min": y_min,
    "y_max": y_max,
}

```

```
def chebyshev(x, k):
```

```

    if k == 0:
        return np.ones_like(x)
    if k == 1:
        return 2 * x - 1

    z = 2 * x - 1
    t_prev2 = np.ones_like(x)
    t_prev1 = z
    for _ in range(2, k + 1):
        t_prev2, t_prev1 = t_prev1, 2 * z * t_prev1 - t_prev2
    return t_prev1

```

```
def solve_lstsq(a, b):
```

```

    return np.linalg.lstsq(a, b, rcond=None)[0]

```

```
def calc_b_values(y):
```

```

    return ((y.max(axis=1) + y.min(axis=1)) / 2.0).reshape(-1, 1)

```

```
def build_tz_matrix(x_groups, degrees):
```

```

    columns = []
    for x_matrix, degree in zip(x_groups, degrees):
        for j in range(x_matrix.shape[1]):
            for k in range(degree + 1):
                columns.append(chebyshev(x_matrix[:, j], k))
    return np.column_stack(columns)

```

```
def split_lambda_coefficients(lambdas, dims, degrees):
```

```

    matrices = []
    start = 0
    for dim, degree in zip(dims, degrees):
        width = dim * (degree + 1)
        matrices.append(lambdas[start:start + width].reshape(dim, degree + 1))
        start += width
    return matrices

```

```

def calculate_psi(lambda_matrix, x, degree):
    psi = np.zeros((x.shape[0], x.shape[1]))
    for j in range(x.shape[1]):
        for k in range(degree + 1):
            psi[:, j] += lambda_matrix[j, k] * chebyshev(x[:, j], k)
    return psi

def incoherence(y_pred, y_true):
    return np.sqrt(np.mean((y_pred - y_true) ** 2, axis=0))

def denormalize(matrix, mins, maxs):
    return matrix * (maxs - mins) + mins

def build_comparison_table(y_names, y_real, y_pred):
    header = ["q"]
    for name in y_names:
        header.extend([f"{name}_real", f"{name}_pred"])

    rows = []
    for idx in range(y_real.shape[0]):
        row = [str(idx + 1)]
        for column in range(y_real.shape[1]):
            row.extend([
                f"{y_real[idx, column]:.6f}",
                f"{y_pred[idx, column]:.6f}",
            ])
        rows.append(row)
    return header, rows

def print_matrix(title, matrix, row_labels=None, col_labels=None, decimals=6):
    print(f"\n{title}")
    matrix = np.atleast_2d(matrix)

    if row_labels is None:
        row_labels = [str(i + 1) for i in range(matrix.shape[0])]
    if col_labels is None:
        col_labels = [f"c{i + 1}" for i in range(matrix.shape[1])]

    formatted = [[f"{value:.{decimals}f}" for value in row] for row in matrix]
    widths = []
    for idx, label in enumerate(col_labels):
        widths.append(max(len(label), max(len(row[idx]) for row in formatted)))
    row_label_width = max(len("row"), max(len(label) for label in row_labels))

```

```

header = " " * (row_label_width + 3) + " ".join(
    label.rjust(widths[idx]) for idx, label in enumerate(col_labels)
)
print(header)
for label, row in zip(row_labels, formatted):
    values = " ".join(value.rjust(widths[idx]) for idx, value in enumerate(row))
    print(f"{label.rjust(row_label_width)} | {values}")

def print_table(title, header, rows):
    print(f"\n{title}")
    widths = [len(name) for name in header]
    for row in rows:
        for idx, value in enumerate(row):
            widths[idx] = max(widths[idx], len(value))

    line = " | ".join(header[idx].rjust(widths[idx]) for idx in range(len(header)))
    print(line)
    print("-+-. ".join("-" * width for width in widths))
    for row in rows:
        print(" | ".join(row[idx].rjust(widths[idx]) for idx in range(len(row))))

def wrap_expression(prefix, terms, indent=" ", terms_per_line=4):
    lines = []
    current = prefix
    for idx, term in enumerate(terms):
        separator = "" if idx == 0 else " + "
        candidate = current + separator + term
        if idx > 0 and (idx % terms_per_line == 0):
            lines.append(current)
            current = indent + term
        else:
            current = candidate
    lines.append(current)
    return "\n".join(lines)

def build_component_formula(output_index, group_index, var_names, lambda_matrix, a_vector):
    terms = []
    for row_idx, var_name in enumerate(var_names):
        for degree in range(lambda_matrix.shape[1]):
            coefficient = a_vector[row_idx] * lambda_matrix[row_idx, degree]
            terms.append(f"{coefficient:.8f} T*({var_name}^{degree})")
    return wrap_expression(f"Φ{output_index + 1}{group_index}(x) = ", terms)

def build_final_formula(output_index, c_matrix):

```

```

terms = [
    f"{c_matrix[group_idx, output_index]:.15f}*Φ{output_index + 1}{group_idx + 1}"
    for group_idx in range(c_matrix.shape[0])
]
return wrap_expression(f"Φ{output_index + 1}(x) = ", terms, indent="    ", terms_per_line=3)

def fit_model(x1, x2, x3, y, p1, p2, p3):
    tz_matrix = build_tz_matrix((x1, x2, x3), (p1, p2, p3))
    b_vector = calc_b_values(y)
    lambda_all = solve_lstsq(tz_matrix, b_vector)
    lambda_x1, lambda_x2, lambda_x3 = split_lambda_coefficients(
        lambda_all,
        (x1.shape[1], x2.shape[1], x3.shape[1]),
        (p1, p2, p3),
    )

    psi_1 = calculate_psi(lambda_x1, x1, p1)
    psi_2 = calculate_psi(lambda_x2, x2, p2)
    psi_3 = calculate_psi(lambda_x3, x3, p3)

    a1 = solve_lstsq(psi_1, y)
    a2 = solve_lstsq(psi_2, y)
    a3 = solve_lstsq(psi_3, y)

    phi_1 = psi_1 @ a1
    phi_2 = psi_2 @ a2
    phi_3 = psi_3 @ a3

    c_matrix = np.zeros((3, y.shape[1]))
    y_pred = np.zeros_like(y)
    for idx in range(y.shape[1]):
        current = np.column_stack((phi_1[:, idx], phi_2[:, idx], phi_3[:, idx]))
        c_matrix[:, idx] = solve_lstsq(current, y[:, idx]).ravel()
        y_pred[:, idx] = current @ c_matrix[:, idx]

    return {
        "tz_matrix": tz_matrix,
        "b_vector": b_vector,
        "lambda_x1": lambda_x1,
        "lambda_x2": lambda_x2,
        "lambda_x3": lambda_x3,
        "a1": a1,
        "a2": a2,
        "a3": a3,
        "phi_1": phi_1,
        "phi_2": phi_2,
        "phi_3": phi_3,
    }

```

```

    "c_matrix": c_matrix,
    "y_pred": y_pred,
    "rmse": incoherence(y_pred, y),
}

```

```
def print_results(data, results):
```

```

    print("ВІДНОВЛЕННЯ ФУНКЦІОНАЛЬНИХ ЗАЛЕЖНОСТЕЙ")
    print("Метод вибору b_iq0: один спільний вектор b для всіх функцій Y_i")
    print("Метод розв'язання: метод найменших квадратів (МНК)")
    print("Нормалізація X: поколоночна, нормалізація Y: спільний діапазон для всіх виходів")
    print("Коефіцієнти  $\lambda$  є спільними для всіх Y_i, окремими є лише A та c")

```

```

    print("\n1. Формуємо вектор b на нормалізованих даних")
    b_rows = [f"q{idx + 1}" for idx in range(results["b_vector"].shape[0])]
    print_matrix("Вектор b", results["b_vector"], row_labels=b_rows, col_labels=["b"], decimals=6)

```

```

    print("\n2. Знаходимо спільні коефіцієнти  $\lambda$  для кожної групи змінних")
    lambda_cols = [f"T{k}" for k in range(results["lambda_x1"].shape[1])]
    print_matrix("Lambda для X1", results["lambda_x1"], row_labels=data["x1_names"], col_labels=lambda_cols)
    print_matrix("Lambda для X2", results["lambda_x2"], row_labels=data["x2_names"], col_labels=lambda_cols)
    print_matrix("Lambda для X3", results["lambda_x3"], row_labels=data["x3_names"], col_labels=lambda_cols)

```

```

    print("\n3. Знаходимо матриці A1, A2, A3")
    print_matrix("Матриця A1", results["a1"].T, row_labels=data["y_names"], col_labels=data["x1_names"])
    print_matrix("Матриця A2", results["a2"].T, row_labels=data["y_names"], col_labels=data["x2_names"])
    print_matrix("Матриця A3", results["a3"].T, row_labels=data["y_names"], col_labels=data["x3_names"])

```

```

    print("\n4. Виразимо частинні функції  $\Phi_k(x)$  через поліноми Чебишева")
    for output_index, y_name in enumerate(data["y_names"]):
        print(f"\nДля {y_name}:")
        print(build_component_formula(output_index, 1, data["x1_names"], results["lambda_x1"], results["a1"][:,
output_index]))
        print(build_component_formula(output_index, 2, data["x2_names"], results["lambda_x2"], results["a2"][:,
output_index]))
        print(build_component_formula(output_index, 3, data["x3_names"], results["lambda_x3"], results["a3"][:,
output_index]))

```

```

    print("\n5. Тепер виразимо всі функції  $\Phi_i(x)$  через  $\Phi_{i1}$ ,  $\Phi_{i2}$ ,  $\Phi_{i3}$ ")
    for output_index in range(len(data["y_names"])):
        print(build_final_formula(output_index, results["c_matrix"]))

```

```

    print_matrix("Матриця C", results["c_matrix"].T, row_labels=data["y_names"], col_labels=["c1", "c2", "c3"])

```

```

    print("\n6. Порівняємо реальні та відновлені значення")
    y_real = denormalize(data["y"], data["y_min"], data["y_max"])
    y_pred = denormalize(results["y_pred"], data["y_min"], data["y_max"])
    header, rows = build_comparison_table(data["y_names"], y_real, y_pred)

```

```

print_table("Порівняння реальних і відновлених значень Y", header, rows)

residual_rows = []
for idx, name in enumerate(data["y_names"]):
    residual_rows.append([
        name,
        f"{results['rmse'][idx]:.6f}",
        f"{np.sqrt(np.mean((y_pred[:, idx] - y_real[:, idx]) ** 2)):.6f}",
    ])
print_table("Нев'язка", ["Y", "RMSE_norm", "RMSE_real"], residual_rows)

def plot_results(y_true, y_pred, y_names):
    fig, axes = plt.subplots(y_true.shape[1], 1, figsize=(9, 4 * y_true.shape[1]), sharex=True)
    if y_true.shape[1] == 1:
        axes = [axes]

    rmse_norm = np.sqrt(np.mean((y_pred - y_true) ** 2, axis=0))
    y_min_common = min(float(y_true.min()), float(y_pred.min()))
    y_max_common = max(float(y_true.max()), float(y_pred.max()))
    padding = 0.03 * (y_max_common - y_min_common if y_max_common != y_min_common else 1.0)

    x_axis = range(1, y_true.shape[0] + 1)
    for idx, ax in enumerate(axes):
        ax.plot(x_axis, y_true[:, idx], color="darkblue", linewidth=2, marker="o", markersize=3, label=f"{y_names[idx]} (норм.)")
        ax.plot(x_axis, y_pred[:, idx], color="tab:red", linewidth=1.8, linestyle="--", label=f"Φ{idx + 1} (норм.)")
        ax.set_title(f"Φ: {y_names[idx]}. Нев'язка: {rmse_norm[idx]:.6f}")
        ax.set_ylim(y_min_common - padding, y_max_common + padding)
        ax.set_ylabel("norm")
        ax.legend(loc="upper right")
        ax.grid(alpha=0.3)

    axes[-1].set_xlabel("q")
    plt.tight_layout()
    plt.show()

```